

ClauseGuard — Tech Stack v3 & Implementation Design

User-defined stack · self-hosted, no managed cloud services · Draft 28 September 2026

1. Tech Stack

#	Component	Choice	Role in the system
1	API	FastAPI	Upload endpoint, question endpoint, health checks. Owns no verdict logic — it routes and returns.
2	File storage	MinIO (Docker Desktop)	Stores the original PDF/DOCX bytes. Self-hosted, S3-compatible API, zero license cost.
3	Message queue	RabbitMQ	Queues each new document picked up from the watched folder so ingestion runs off the request path.
4	Relational database	PostgreSQL	Documents, chunks, findings, users, job status, version records. Single source of truth.
5	Vector store + lexical search + fusion	Qdrant (dense + sparse vectors), RRF	One engine for both semantic (dense) and keyword-style (sparse/BM25) search, fused with Reciprocal Rank Fusion. Removes the need for a second search engine.
6	Background jobs	Celery workers (consuming RabbitMQ)	Actually does the parsing, embedding, and indexing work queued in step 3.
7	Graph store	Neo4j	Stores relationships between clauses: definitions, cross-references, amendments that supersede earlier clauses.
8	Reranker	Cross-encoder model	Re-orders the top candidates from Qdrant so the most relevant ones are actually first before the LLM sees them.
9	Embedding model	OpenAI <code>text-embedding-3-small</code>	Converts chunk text and questions into vectors for semantic search.
10	Relevance grading	CRAAG-style grader	Checks retrieved evidence before generation. Good evidence → answer. Weak evidence → try again. No evidence → abstain.
11	Query expansion	HyDE	Always on. Expands short/vague questions into fuller hypothetical answers before retrieval, to close the wording gap with dense contract language.
12	Cache	Redis	Caches hot embeddings and frequently repeated queries so they don't re-hit Qdrant/the embedding API every time.
13	User management	PostgreSQL (email/password + JWT sessions)	No OIDC/SAML. Users, roles, and sessions are plain tables — simplest thing that works for a self-hosted single-deployment setup.
14	Metadata filtering	Postgres columns + Qdrant payload	Tenant, document family, and version are filtered before ranking, not after — applied inside the Qdrant query itself.

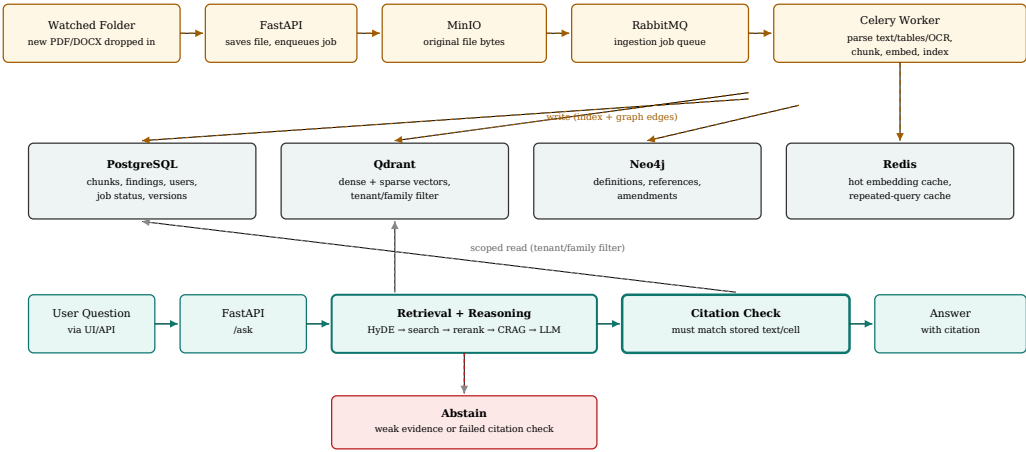
Removed from the earlier design, on request:

- **OpenSearch** — Qdrant's sparse-vector (BM25-style) search covers lexical matching, so a second search engine isn't needed.
- **Kafka** — RabbitMQ is a simpler, lighter queue and is enough for a single-deployment, folder-watch ingestion pattern. Kafka's replay/partition-log strengths matter at a scale and multi-consumer complexity this setup doesn't have yet.
- **Temporal** — Celery is enough to run background ingestion jobs with retries. Temporal's durable-workflow guarantees matter once ingestion logic gets very stateful (many resumable steps, long-running sagas) — worth revisiting later, not needed to start.

Two gaps to decide before build, not covered by your list above:

1. **Generation model** — item 9 is the *embedding* model only. You still need an LLM to actually answer the question (propose a span/cell + verdict). Any provider works behind an adapter (OpenAI, Anthropic, local) — pick one and pin it, since it gets version-stamped on every answer.
2. **Citation verification** — the step that checks the LLM's claimed quote against the real stored text/cell before it's allowed to reach the user. Without this step, "grounded" is just a claim, not a guarantee. Recommend keeping it as a mandatory, non-skippable step regardless of stack changes.

2. Updated Architecture



■ **async:** folder watch → parse → index ■ **sync:** question → verified answer ■ **scoped read** ■ **abstain**

3. Ingestion Phase — How a Document Gets In

In plain terms: a file lands in a folder, gets stored safely, then gets broken into searchable pieces in the background — the user never waits for this.

1. **1 Watch the folder.** A small watcher process notices a new file appear and hands it to FastAPI.
2. **2 Save the original file first.** FastAPI uploads the raw bytes to MinIO before anything else happens. This is the copy the reviewer will eventually see highlighted — it never gets altered.
3. **3 Record it and queue it.** FastAPI writes one row to PostgreSQL ("this document exists, status = received") and puts one job on RabbitMQ, in the same step — so a job is never queued for a file that failed to save, and a saved file never gets silently forgotten.
4. **4 A Celery worker picks up the job.** It reads the file back from MinIO and starts parsing.
5. **5 Parse the content.** Plain paragraphs become text chunks with page numbers. Tables become individual cells, each one tagged with its row and column header — including multi-level headers (e.g. "Fee Schedule → Tier 2 → Monthly Rate"), so a cell is never just "some number near a table." Scanned pages go through OCR. Anything that fails to parse cleanly is marked `unparsed` instead of guessed.
6. **6 Find the relationships.** The worker also looks for cross-references: "this clause references Schedule A," "this term is defined in Section 1," "Amendment 3 changes Section 6." These become edges in Neo4j — separate from the text itself.
7. **7 Turn text into vectors.** Each chunk and each table cell (as a natural-language sentence, e.g. "the Monthly Rate for Tier 2 is \$X") gets embedded using the small OpenAI embedding model.
8. **8 Index everything.** Chunks, cells, and vectors go into Qdrant (with tenant/document/family tags attached so later searches can be scoped). The graph edges go into Neo4j. Everything is cross-linked back to the PostgreSQL row for that document.
9. **9 Mark it ready.** The PostgreSQL row updates to "indexed." Only now can this document show up in an answer.

If a worker crashes mid-file, the job stays on RabbitMQ (or gets retried by Celery) — the document simply stays in "received" or "partially indexed" status until it's picked up again. Nothing is lost, nothing is double-counted, because indexing is keyed to the document id.

4. Query & Retrieval Phase — How a Question Gets Answered

In plain terms: the question is expanded, searched from two angles at once, narrowed down, checked for relevance, answered, and then fact-checked against the original document before anything is shown.

4.1 Step by step

1. **1 Question comes in.** FastAPI receives it along with which tenant/document family it applies to.
2. **2 Check the cache.** Redis is checked first — if this exact question (or a very close match) was answered recently for the same scope, and nothing has changed, reuse it instead of redoing the work.
3. **3 Expand the question (HyDE).** The question is short; contracts are written in dense legal language. Before searching, the system asks the LLM to sketch a hypothetical answer in contract-like wording, and searches with that expanded version too — this closes the gap between how people ask and how contracts are written.
4. **4 Search two ways at once, scoped.** Qdrant is queried twice in the same call: once as a meaning-based (dense vector) search, once as a keyword-based (sparse/BM25-style) search. Both searches are restricted to the right tenant/family/version *before* ranking — so results from another tenant or an old contract version are never even considered.
5. **5 Pull in related clauses from the graph.** Neo4j is checked for one hop out from the top candidates — does this clause depend on a definition, a schedule, or get changed by a later amendment? If so, that related chunk is added to the candidate list.
6. **6 Combine the result lists.** The dense list, the sparse list, and the graph results are merged with Reciprocal Rank Fusion — a simple, well-tested way to combine differently-scored lists into one ranked list.
7. **7 Re-rank the top handful.** The cross-encoder reranker looks closely at the top ~20-50 candidates and reorders them so the most genuinely relevant ones are at the very top — this step is slower but only runs on a short list.
8. **8 Grade the evidence (CRAG).** Before generating anything, the system checks: is this evidence actually good enough to answer from?
 - **Good** → move to step 9.
 - **Unclear** → rewrite the question once and repeat steps 4-7 with the rewrite.
 - **Bad** → stop here and abstain. Skip the expensive generation step entirely.
9. **9 Generate the answer.** The LLM sees only the top, verified-relevant evidence and the question. It responds in a fixed format: which exact span or table cell supports the answer, and what the answer is. No free-form quoting, no tool use, no randomness (temperature 0).
10. **10 Check the citation.** Before the answer is shown to anyone, the system independently checks: does the span or cell the LLM pointed to actually say what it claims, byte-for-byte against what's stored? If yes, the answer ships. If no, retry generation (up to 3 times) — if it still fails, abstain rather than show an unverified answer.
11. **11 Cache and return.** A verified answer is cached in Redis (keyed to the question + scope + document version, so a later document update invalidates it) and returned with its citation.

4.2 Why the order matters

Rule	Reason
Filter before ranking, not after	If you rank first and filter second, a true match from the right tenant can get pushed out of the top-k by irrelevant results from other tenants before the filter ever gets a chance to remove them.
Grade before generating	Running the LLM on evidence that's already known to be weak wastes time and money, and produces answers that only look confident.
Verify after generating, always	The LLM can describe a citation that sounds right but isn't. This is the one check that turns "the model said so" into "the document actually says so" — it must never be skipped, regardless of which other stages are on or off.